

.NET Architectural Blueprint

MVC Design Pattern, Monolithic Architecture & Microservices Paradigm

1. ASP.NET Core MVC Architecture

Model-View-Controller (MVC) is a UI-level architectural pattern that enforces Separation of Concerns by dividing application responsibilities into three components:

- **Model:** Represents the domain data structures, state, and business logic validation rules (e.g., C# classes/records).
- **View:** Responsible for user interface rendering and presenting dynamic data to the user (e.g., Razor `.cshtml` files).
- **Controller:** Handles incoming HTTP requests, processes input parameters, coordinates operations via business services, and returns appropriate HTTP responses or Views.

ASP.NET Core Controller Example:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductService _service;
    public ProductsController(IProductService service) => _service = service;

    [HttpGet("{id:int}")]
    public async Task<ActionResult> GetById(int id, CancellationToken ct)
    {
        var product = await _service.GetProductByIdAsync(id, ct);
        return product is not null ? Ok(product) : NotFound();
    }
}
```

2. Monolithic Architecture in .NET

A **Monolith** packages all application components (UI, API endpoints, business logic, data access layer) into a single code base and single deployment unit (e.g., one executable or DLL set running on a single server/container).

- **Advantages:** Simple initial development, easy end-to-end debugging, simple single-artifact deployment, and zero network latency for inter-module calls.
- **Disadvantages:** Tightly coupled codebases grow hard to scale independently, risk total service outage if one component crashes, and demand long build times as teams expand.
- **Modular Monolith:** A hybrid approach structuring a single codebase into well-defined, isolated domain boundaries/assemblies (e.g., Clean Architecture), paving a clean migration path toward microservices.

3. Microservices Architecture in .NET

Microservices decompose an application into autonomous, independently deployable services organized around discrete business capabilities (Bounded Contexts). Each microservice owns its business logic and data store.

- **Independent Deployment & Scaling:** Scale individual services horizontally based on targeted load (e.g., high traffic on Order service vs low traffic on Billing).
- **Resilience & Isolation:** A failure in one microservice does not cascade to take down the entire system.

- **Inter-Service Communication:** Uses synchronous HTTP/gRPC for queries and asynchronous message brokers (RabbitMQ, Azure Service Bus, Kafka) for domain events.

4. Monolith vs. Microservices Comparison

Attribute	Monolithic Architecture	Microservices Architecture
Deployment Unit	Single unified application package	Multiple independent services/containers
Data Storage	Single central database	Database per service pattern
Communication	In-memory method calls	gRPC, REST HTTP APIs, Async Message Queues
Operational Complexity	Low (Simple CI/CD pipeline)	High (Requires Kubernetes, API Gateways, Tracing)